

CMPE 492

Digitization of Historical Meteorology Graphs from
the Kandilli Observatory Archive

Ahmet Selçuk Ersoy

Kerem Bozkurt

Advisor:

H. Birkan Yılmaz

GitHub Repository:

<https://github.com/kerembozkurt2002/>

`Digitization-Of-Historical-Meteorology-Graphs`

June, 2026

TABLE OF CONTENTS

1. INTRODUCTION	1
1.1. Broad Impact	2
1.2. Ethical Considerations	2
2. PROJECT DEFINITION AND PLANNING	4
2.1. Project Definition	4
2.2. Project Planning	4
2.2.1. Project Time and Resource Estimation	4
2.2.2. Success Criteria	5
2.2.3. Risk Analysis	5
2.2.4. Team Work	5
3. RELATED WORK	8
4. METHODOLOGY	9
5. REQUIREMENTS SPECIFICATION	11
6. DESIGN	13
6.1. Information Structure	13
6.2. Information Flow	13
6.3. System Design	14
6.4. User Interface Design	15
7. IMPLEMENTATION AND TESTING	17
7.1. Implementation	17
7.2. Testing	18
7.3. Deployment	19
8. RESULTS	21
9. CONCLUSION	23
REFERENCES	24
APPENDIX A: CSV EXPORT SCHEMA	26
APPENDIX B: CALIBRATION JSON SCHEMA	28
APPENDIX C: PER-TEMPLATE COLOR PROFILES	30

1. INTRODUCTION

The Kandilli Observatory has been recording atmospheric measurements continuously since the late nineteenth century. The temperature record between 1940 and 1990 survives almost exclusively as paper thermograms — continuous ink traces drawn by a mechanical pen on a rotating drum chart. The observatory’s archive contains roughly 1,682 high-resolution TIFF scans of these charts, organized by year, chart format, and month, but the actual numerical time series the charts represent has never been captured in machine-readable form. Without that step the data cannot be used in climate reanalysis, statistical studies, or modern visualizations.

The goal of this project is to build a desktop application that turns each scanned thermogram chart into a clean (`datetime`, `temperature °C`) time series ready for downstream analysis. The application is shipped as a single binary, supports macOS and Windows, and is designed to be operated by a researcher with domain knowledge but no programming background. The workflow is human-in-the-loop: an image-processing pipeline produces an initial curve extraction, and a focused editing UI lets the user inspect the overlay, correct individual points, refine specific regions, or redraw a damaged portion of the curve before exporting CSV.

Three properties of the data make automation hard. First, the temperature curve is extremely thin — approximately 0.1–0.5 % of pixels — against a dominant printed grid the curve constantly crosses. Second, the chart was printed on a cylindrical drum, so the vertical grid lines on the scan are not straight but follow a barrel-distorted parabola. Third, the dataset spans five decades and three chart formats (Daily, 4-day, Weekly) with at least nine visually distinct templates that differ in ink color, paper aging, and grid color (orange, green, yellow, pink). A single fixed pipeline cannot handle all of them, so the system was built around per-template color and geometry profiles selected automatically at upload time.

1.1. Broad Impact

The immediate beneficiary is the climate-science community working on the Eastern Mediterranean. A continuous, structured daily temperature record covering the second half of the twentieth century is a building block for trend analysis, validation of reanalysis products, and downscaling of regional climate models. Beyond the specific dataset, the digitization workflow we describe is directly transferable to similar paper archives held by national meteorological services, historical observatories, and citizen-science data-rescue projects. The system replaces a manual transcription task estimated at several minutes per chart with a per-chart workflow of well under a minute for clean scans, with manual editing reserved for the difficult cases. At the dataset scale of 1,682 charts the speed-up is the difference between a feasible undergraduate project and a multi-year manual effort.

A secondary impact is methodological. The choice to use classical computer vision instead of a learned segmentation model was deliberate: training data does not exist, the dataset is too heterogeneous to label exhaustively, and a transparent pipeline can be inspected and corrected by a domain expert. The project demonstrates that careful classical CV plus a well-designed editing UI can meet the accuracy requirements of historical-data rescue without the overhead of model training, GPU infrastructure, or the explainability problems of a neural network.

1.2. Ethical Considerations

The raw chart scans are the property of the Kandilli Observatory and are not redistributed by this project; the application processes images supplied by the user and writes derivative CSV files next to the source image. No identifying or personal information is contained in the data — the only signal recorded by the instrument is air temperature — so privacy concerns are minimal. The principal ethical risks are scientific rather than personal. An automatically extracted time series carries an implicit claim of accuracy that downstream users may overestimate. We mitigate this in three ways: every extracted point preserves its source pixel coordinates so the

original measurement can always be re-derived; the CSV export marks every point that has been manually edited so downstream analyses can filter on provenance; and the application requires explicit per-template calibration by a human before extraction is permitted, so silent miscalibration is unlikely.

2. PROJECT DEFINITION AND PLANNING

2.1. Project Definition

The project delivers a cross-platform desktop application that takes a scanned thermogram chart (TIFF, PNG, or JPEG) and produces a CSV file of (`id`, `time`, `temperature`, `x_pixel`, `y_pixel`, `is_edited`) records sampled along the temperature curve. The user-facing flow has four stages: (i) upload a scan, automatically classify it into one of nine templates, and reload the last calibration if one exists; (ii) calibrate the grid geometry once per template using a seven-step interactive procedure; (iii) extract the curve in the chosen view, optionally constraining the extraction with starting/ending markers or by drawing a hint stroke; (iv) review, edit, and export. The application bundles a Python sidecar that runs the image-processing pipeline; the Tauri/Rust host process exposes the pipeline through a small set of commands and is responsible for file I/O, temp-file lifecycle, and OS integration.

The deliverable is the application source plus the calibration files for all nine templates we identified in the archive. The project does not ship the original scans, does not perform climatological analysis on the extracted data, and does not provide a cloud or multi-user backend — the application runs entirely on the researcher’s machine.

2.2. Project Planning

2.2.1. Project Time and Resource Estimation

Active development ran from 4 February 2026 to 4 June 2026, which is roughly eighteen weeks. The two team members worked in parallel, splitting roughly equally between Python image-processing work and TypeScript/Rust application work, with shared time spent on architecture, dataset classification, and weekly advisor meetings. The hardware requirements are modest: a laptop with 8 GB of RAM and a recent

macOS or Windows installation is sufficient. The third-party software dependencies are all open source: Python 3.11+, OpenCV [1], NumPy [2], Tauri 2 [3], React [4], and Zustand [5]. No paid services, no GPU, and no cloud infrastructure are required.

2.2.2. Success Criteria

We adopted three criteria, in order of importance.

- (i) **Faithful extraction on a clean scan.** On a representative scan with intact ink and a clearly printed grid, the extracted curve, when overlaid back on the image, must lie on the ink trace at visual inspection across the full time span of the chart. This is the bar that says the pipeline is working at all.
- (ii) **Recoverable extraction on a degraded scan.** On a scan with stains, fading, handwritten notations, or partial loss of ink, the application must allow a domain expert to reach the same final accuracy as the clean case within a few minutes of editing — through point-level drag, region refinement, or freehand redraw with snap-to-curve.
- (iii) **Reproducible export.** Re-extracting the same image with the same calibration must produce the same CSV bit-for-bit, and the CSV must mark every manually edited point so downstream users can filter on provenance.

2.2.3. Risk Analysis

Five risks were identified at the start of the project and tracked throughout. Table 2.1 summarizes them along with the mitigation that was adopted.

2.2.4. Team Work

Both authors were equal contributors and held joint responsibility for all major architectural decisions. The day-to-day split was vertical — each person owned a coherent slice of the stack so that pull requests rarely conflicted — with shared ownership of integration, testing, and writing. Table 2.2 summarizes the split.

Table 2.1. Risk register and adopted mitigations.

Risk	Prob.	Impact	Mitigation
Curve too thin to extract reliably	High	High	Per-template color profiles; sliding-window Y tracking with median consensus; MAD outlier removal.
Curved grid mis-aligns the time axis	High	High	Cylindrical model $x = \text{linearX} + a(y - c_y)^2$ fit during calibration; per-template calibration file.
Heterogeneous templates break a single pipeline	High	Medium	Automatic template classification at upload; nine independent color profiles and calibration files.
Cross-platform packaging issues	Medium	Medium	Tauri sidecar pattern; tested on macOS first, ported to Windows in last sprint.
User error during calibration silently corrupts output	Medium	High	Seven-step guided modal; alignment-only re-entry that reuses existing calibration; CSV marks every edited point.

Table 2.2. Vertical responsibility split. All architecture and process work was shared.

Area	Ahmet Selçuk Ersoy	Kerem Bozkurt
Desktop host (Tauri/Rust) and IPC layer	Primary	Reviewer
React UI, workspace, modals, drag-drop	Primary	Reviewer
Zustand stores, edit history, CSV export	Primary	Reviewer
Python pipeline core (preprocess, segmenter)	Reviewer	Primary
Template detector and per-template profiles	Reviewer	Primary
Calibration processor and geometry math	Reviewer	Primary
Dataset classification (nine templates)	Joint	Joint
Architecture, planning, advisor meetings	Joint	Joint
Testing and integration	Joint	Joint
Report writing	Joint	Joint

3. RELATED WORK

The chart-digitization problem has been studied at three levels of generality. At the most general level are open-source tools such as WebPlotDigitizer [6] and Engauge Digitizer [7], which let a user calibrate two axes by clicking and then mark points on a plotted curve. They are excellent for one-off figures but assume a printed chart with a straight grid, a single short curve, and a willing human ready to click every point. They do not scale to a 1,682-image archive.

At the next level are dedicated systems for scientific-document figure mining. These typically combine vector-graphics extraction from PDFs with deep-learning recognition of axes and curves. They target born-digital figures, not scanned analog instrument output, and their assumptions about clean grids and well-defined legends do not transfer.

At the most specific level is the historical-climate data-rescue literature. The IEDRO initiative [8] and the BAMS inventory by Brönnimann et al. [9] catalogue archives in the same condition as Kandilli’s: paper records, multiple chart formats, decades of drift. The community has converged on a human-in-the-loop workflow — automate where possible, expose the result for review, capture corrections — which is exactly the model this project adopts.

On the algorithmic side, the pipeline reuses classical building blocks: CLAHE for contrast normalization [10], bilateral filtering for edge-preserving smoothing [11], the Hough line transform for grid detection [12], and MAD for robust outlier rejection [13]. We considered a learned segmentation model (U-Net [14]) but rejected it: training data did not exist, hand-labeling enough thermograms to cover nine templates plus aging variations was out of scope, and a transparent classical pipeline could be calibrated and audited by the user. The interesting contribution of this project is therefore not algorithmic novelty but the per-template profile system, the cylindrical-grid calibration, and the human-in-the-loop editing UI tying them together.

4. METHODOLOGY

The pipeline is six logical stages with one classifier at the front (Figure 4.1). Each stage has a single responsibility and writes its output to a typed dataclass that the next stage consumes.

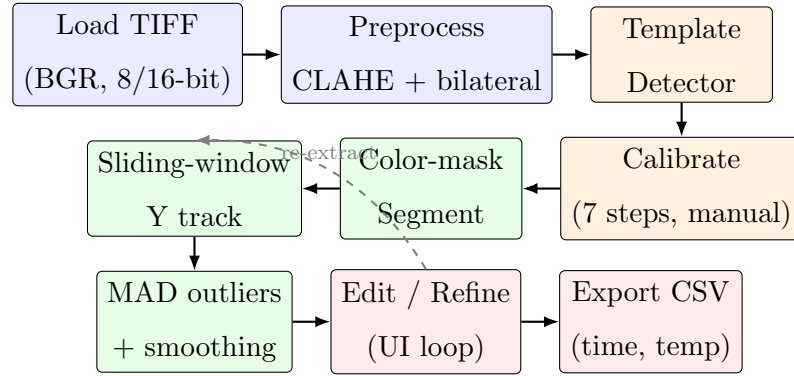


Figure 4.1. End-to-end processing pipeline. Blue stages run automatically on upload; orange stages run once per template and are persisted; green stages run on every curve request; red stages are user-driven.

Preprocessing normalizes color space and bit depth to BGR 8-bit, applies a bilateral filter ($d = 9$, $\sigma_{\text{color}} = \sigma_{\text{space}} = 75$) to denoise without destroying the ink edge, and runs CLAHE in LAB color space (clip 2.0, 8×8 tiles) on the L channel only. The result is then handed to the template detector or, in production, used directly as the input to segmentation.

Template detection converts a 400×120 resize of the scan into a 150-feature vector — aspect ratio buckets, vertical-line density via Sobel + FFT, left/right/top edge variance and Canny edge density, BGR means in five regions, an 18-bin hue histogram in the chart center, three full BGR 24-bin histograms, and corner statistics. A weighted combination of cosine similarity and normalized Euclidean distance against pre-computed signatures for the nine templates picks the best match. The confidence is the best score scaled by its ratio to the runner-up.

Calibration is interactive and persisted per template. The user performs three

horizontal steps (mark top horizontal line, mark its rightmost point for rotation, enter the temperature value and spacing) and four vertical steps (mark a vertical line's top and bottom, enter the reference hour, set the curvature center and slider). The processor stores the rotation angle, horizontal/vertical spacings, line positions, and the curvature coefficient a in the cylindrical model $x = x_{\text{ref}} + a(y - c_y)^2$.

Curve segmentation is the core stage. A binary mask of curve pixels is built using RGB rules tuned per template profile (e.g., for the dark-blue-ink-on-orange-grid **haftalik-2** template, $\text{maxRGB} \leq 120$, saturation between 5 and 100, with a grayscale fallback for pencil-like traces). A sliding window centered on an expected y value walks left to right across the columns, taking the median of mask pixels within a ± 5 pixel tight band when possible and a ± 300 pixel coarse band as fallback; a configurable maximum jump $\Delta y_{\text{max}} = 20$ pixels per column keeps the tracker from leaping to a stamp or handwriting. The output of this stage is a column-indexed raw y vector.

Refinement interpolates over short gaps, builds a wide-window reference curve (moving median width 151, Gaussian $\sigma = 18$, kernel 81), refines each column to lie within ± 40 pixels of the reference, removes outliers using a MAD threshold scaled by 4.0, and smooths the result with a length-3 moving median followed by a length-5 Gaussian. Top- and bottom-clipped columns are snapped to the grid boundary so an out-of-range portion of the curve traces the edge rather than being interpolated diagonally across a gap.

Export converts pixel coordinates to time using the curvature-corrected vertical line model and to temperature by linear interpolation between adjacent horizontal grid lines. The CSV is written next to the source image and contains (`id`, `time`, `temperature`, `xValue`, `yValue`, `isEdited`) — the last column is only emitted when at least one point in the curve has been manually edited.

5. REQUIREMENTS SPECIFICATION

The application supports a single user — a researcher working with the Kandilli archive on their own laptop — and the use-case set is therefore narrow. Figure 5.1 summarizes the relationships.

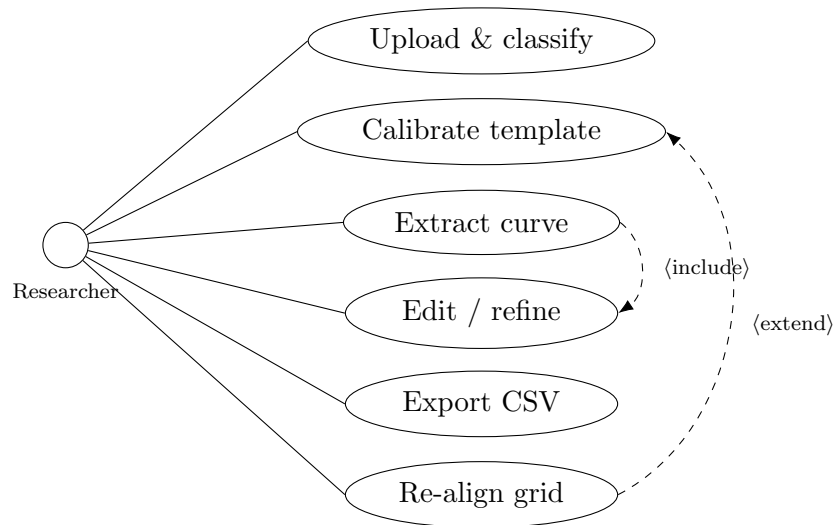


Figure 5.1. Use-case diagram. A single researcher actor drives all six top-level use cases; editing is included by extraction and re-alignment extends calibration.

Functional requirements. The application must accept TIFF, PNG, or JPEG input either through a file dialog or drag-and-drop. It must classify the input into one of the nine templates and surface the confidence and all candidate scores so the user can override. It must require an explicit per-template calibration before allowing extraction; the alignment modal must let the user re-align an existing calibration in place without forcing a full recalibration. It must extract the curve automatically when the user switches to curve view, accept optional segment-level starting and ending markers, accept freehand drawing as a hint that is snapped to the curve, and accept a rectangular refine area to re-extract a local region. It must support per-point dragging, multi-point deletion, undo/redo, and a reset-to-raw action. It must export CSV next to the source image and offer Reveal-in-Finder and Open-with-default-app actions on the result.

Non-functional requirements. The desktop binary must run on macOS and

Windows. A complete extract on a typical 4 Mpx scan must finish in under 3 seconds on a laptop without a discrete GPU. Re-extraction with the same calibration must be deterministic. All temporary files written for rotation correction must be cleaned up on exit and on new-image load. The application must function fully offline.

6. DESIGN

6.1. Information Structure

The application is filesystem-oriented and does not use a relational database. Long-lived state is held in three on-disk artifacts: the source images chosen by the user, one calibration JSON per template under `backend/calibrations/`, and the CSV exports the application writes next to each source image. Short-lived state (the loaded image, the current extraction, the edit history, the drawing strokes, the refine rectangle) lives in four Zustand stores in the frontend. Figure 6.1 shows the entity relationships that exist in the calibration JSON; everything else is per-session.

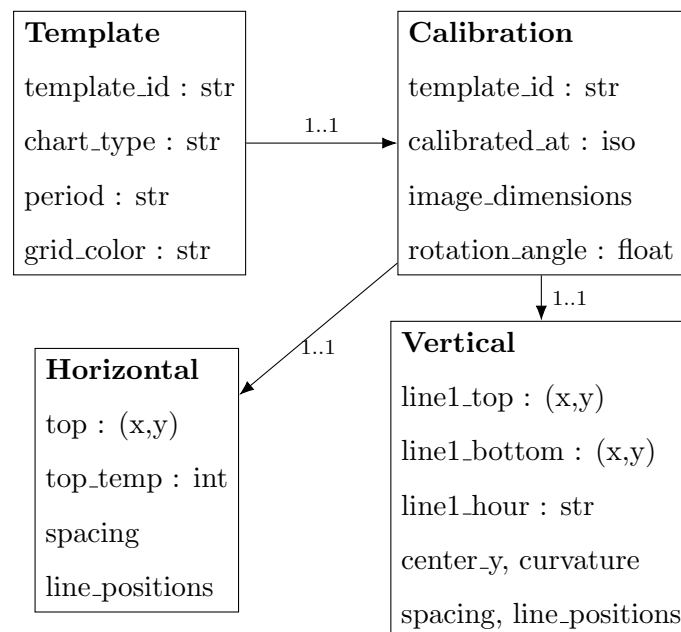


Figure 6.1. Entity model held in each calibration JSON file. The nine templates each have at most one calibration record.

6.2. Information Flow

The interaction between user, frontend, Rust host, and Python sidecar is summarized in Figure 6.2. The host is a thin shell: it receives a Tauri command, spawns `python3 backend/main.py` with the right arguments, and parses the JSON response. All image-processing logic lives in the sidecar.

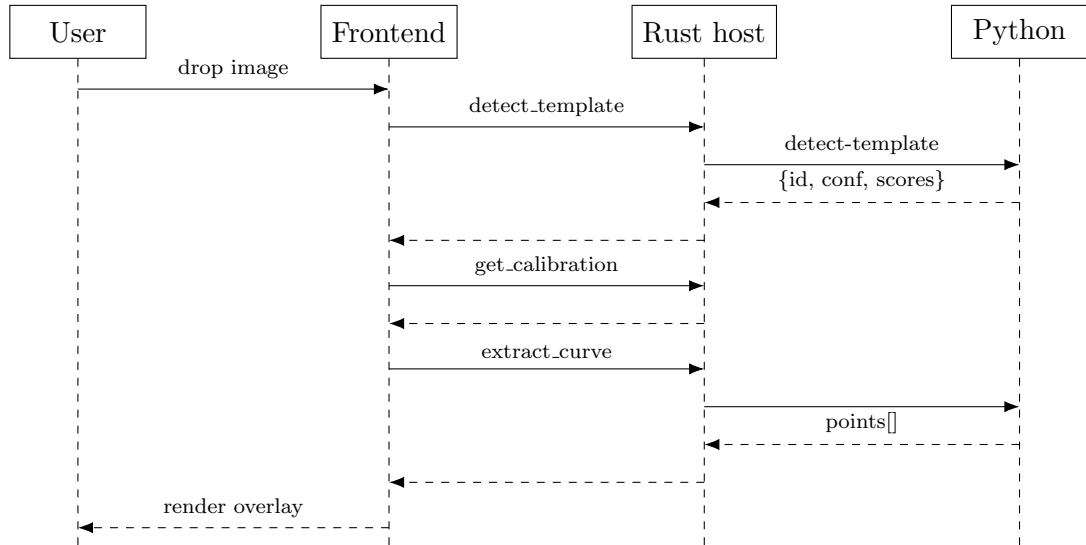


Figure 6.2. Sequence of interactions for the upload-to-extract path. Calibration writes follow the same shape; export bypasses Python and serializes CSV directly in the frontend.

6.3. System Design

The system is partitioned into four cooperating pieces. The React frontend renders the workspace, modals, and overlays. Four Zustand stores hold per-session state: `imageStore` for the loaded scan and detected template, `calibrationStore` for the modal flow, `curveStore` for extracted points, edit history, drawing strokes, refine rectangle, and undo/redo, and `dataStore` for export metadata. The Rust host exposes eleven Tauri commands and owns the temp-file tracker that sweeps rotated PNGs on new-image load and on exit. The Python backend is a CLI with seven subcommands (`preview`, `detect-template`, `get-calibration`, `save-calibration-simple`, `extract-curve`, `snap-drawing`, and the legacy preview); each subcommand reads its arguments, runs one pipeline pass, and prints a JSON document to stdout. The frontend never talks to Python directly — every IPC trip goes through the Rust host, which is also where the temp PNGs produced by the manual-rotation UI are tracked and cleaned up on new-image load and on application exit.

6.4. User Interface Design

The workspace is a three-region layout: a left sidebar with upload, template selector, view buttons (0 Image, 1 Grid, 2 Curve), and a curve panel that appears in curve view; a center canvas that renders the image plus overlays; and a top header with Calibrate and Align actions. Keyboard shortcuts mirror the view buttons (0/1/2), zoom (Ctrl/Cmd +/- /0), and undo/redo (Ctrl/Cmd Z, Ctrl/Cmd Shift Z). Drag-and-drop is handled by Tauri's native drag-drop event so files are accepted without round-tripping through a hidden `<input>` element. Figure 6.3 shows the main workspace; Figure 6.4 shows the calibration modal.

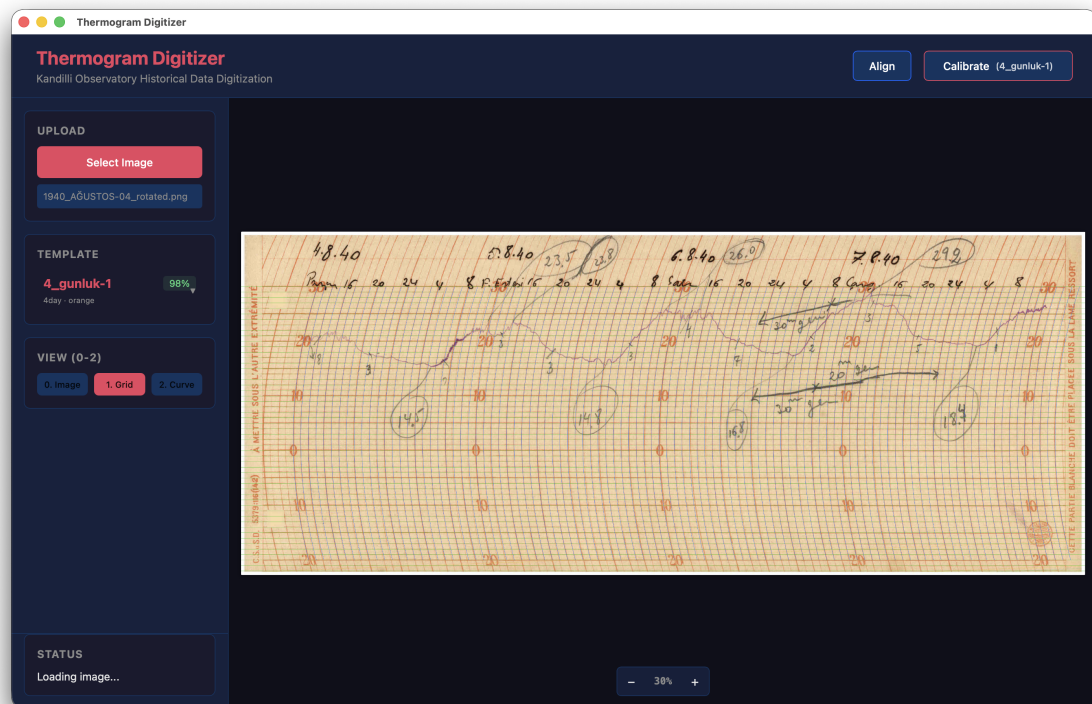


Figure 6.3. Main workspace with grid overlay visible. Left: upload, template, view, and curve panels. Center: image with grid overlay. Top right: Align and Calibrate actions.

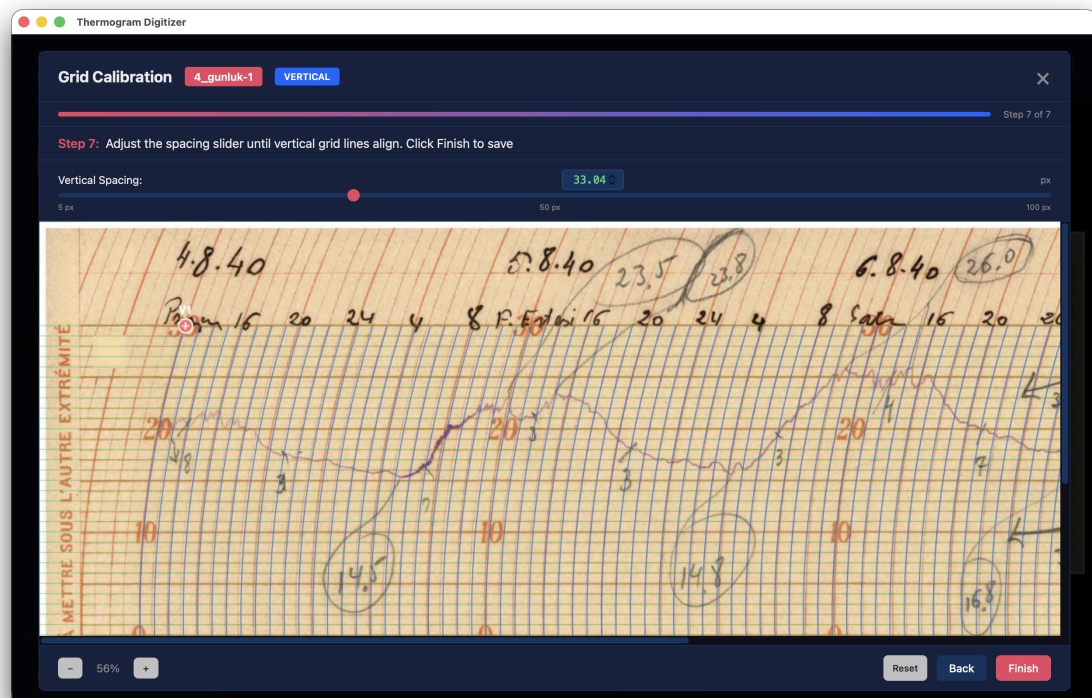
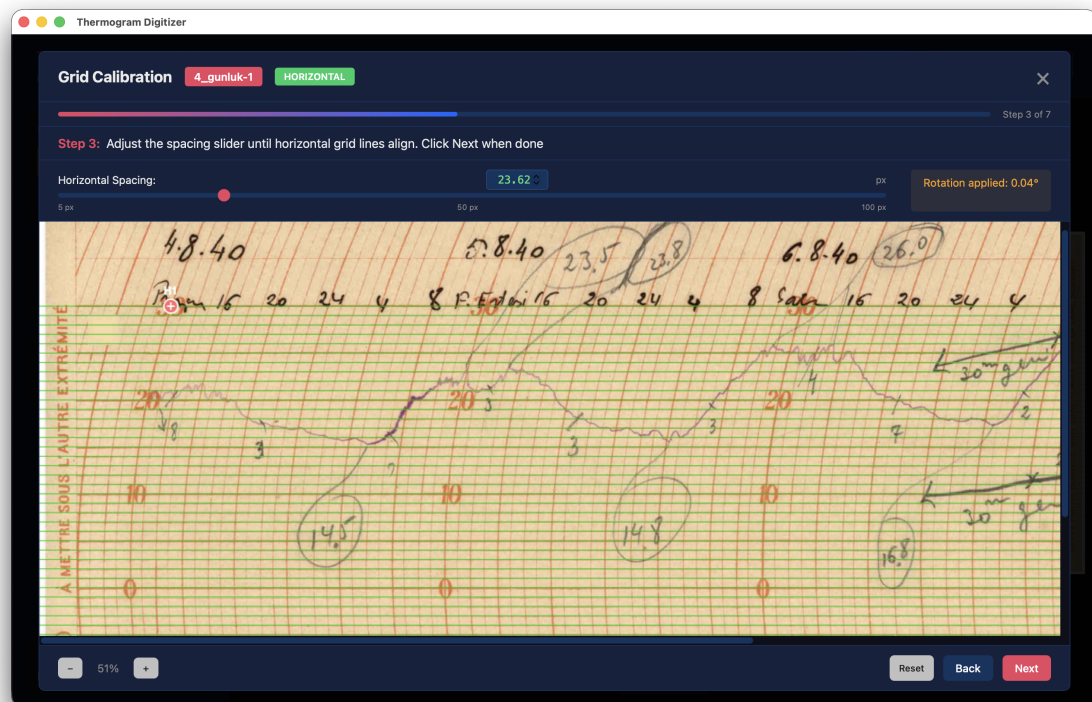


Figure 6.4. Seven-step calibration modal. Top: the three horizontal steps fix rotation, the top temperature value, and the vertical grid spacing. Bottom: the four vertical steps mark a reference line, set the curvature center and coefficient, and fix the horizontal grid spacing.

7. IMPLEMENTATION AND TESTING

7.1. Implementation

The desktop host is Tauri 2.0 with Rust 1.78+; the frontend is React 18 + TypeScript + Vite; the backend is Python 3.11+ with OpenCV 4 and NumPy. The Rust host exposes eleven commands; the most important are `preview_grid`, `detect_template`, `get_calibration`, `save_calibration_simple`, `extract_curve`, `snap_drawing_to_curve`, `save_rotated_image`, `write_csv_next_to_image`, `open_file_path`, `reveal_file_in_folder`, and `cleanup_rotated_temp_files`. Each one performs argument marshaling, spawns `python3 backend/main.py` with the appropriate subcommand, and parses the JSON response into a strongly typed Rust struct that Tauri serializes back to TypeScript.

Color-mask segmentation. Per-template color profiles encode RGB rules tuned by inspecting representative scans. The default profile detects pinkish/reddish ink common to `haftalik` and `4_gunluk` templates; `gunluk-1` switches to a grayscale-friendly profile for faint pencil on a yellow background; `gunluk-2` targets violet ink on a green grid; `gunluk-3` handles a darker pencil-like trace on an orange grid; `haftalik-2` captures nearly black ink on an orange grid with a grayscale fallback. The mask is built using vectorized NumPy boolean operations across the R, G, B channels and a derived saturation, so the per-image cost is dominated by the single `cv2.cvtColor` call rather than the Python loop.

Sliding-window tracker. Once the mask is built, the segmenter walks the columns from $x_{\min}$ to $x_{\max}$, maintaining an `expected_y` that is initialized from a user-supplied y -hint or auto-detected from the first column that has mask pixels. For each column it prefers pixels within a tight ± 5 pixel band around `expected_y` and falls back to a ± 300 pixel coarse band when nothing is close; the column median becomes the tracked value and is clamped to a configurable maximum jump per step. The recent history is held in a small ring buffer and the expected value is refreshed every ten columns by the buffer median. This is the central design choice of the project — it is

what lets the same code follow a curve that crosses the grid hundreds of times without being deflected by either the grid or stray ink.

Cylindrical calibration. The vertical grid lines on a drum-printed chart follow a barrel-distorted parabola, modeled as $x = x_{\text{ref}} + a(y - c_y)^2$. The calibration modal asks the user to mark the top and bottom of a single vertical line, drag a slider for the curvature center c_y and the coefficient a , and read the result on the live overlay. The processor then propagates the same shape to every other grid line by translating x_{ref} along the horizontal axis at the user-supplied spacing. The export path inverts this mapping at every extracted point to recover the true time coordinate before interpolating between adjacent grid lines.

Frontend editing. The curve store keeps two arrays — raw extraction and edited points — and a bounded edit history list that supports undo and redo. The CSV exporter compares the two arrays per point to decide whether to mark a row as edited; if no point differs, the `isEdited` column is omitted entirely so clean exports stay narrow. The refine area, freehand strokes, and starting-point markers are all rendered on a single overlay canvas sized to the natural image dimensions and scaled by the zoom transform.

Temp-file lifecycle. Manual rotation produces a normalized PNG that is written to the OS temp directory by the `save_rotated_image` command. Every such file is recorded in a `TempFileTracker` held as Tauri managed state; on new-image load the tracker is purged so the previous session’s files do not linger, and the same purge runs on the `Exit` run event so app close is clean.

7.2. Testing

Testing happens at two layers. Day-to-day, every pull request is validated interactively by opening at least one representative scan from each of the nine templates, running calibration if needed, switching to curve view, and confirming the overlay matches the ink trace; cross-platform packaging is checked the same way on

macOS and Windows. Two batch comparison scripts under `backend/scripts/` — `batch_extract_compare.py` and `batch_profile_compare.py` — run the segmenter across folders of images and emit side-by-side overlays for visual regression whenever a color profile or segmenter parameter changes; this was the workhorse for tuning the per-template profiles.

Underneath the interactive layer the Python backend ships a fifteen-file pytest suite (`backend/tests/`) with sixty-six cases, all synthetic so the suite has no runtime dependency on real scans or pre-saved calibration files. Shared fixtures in `conftest.py` construct a blank image, a noisy image, a horizontal pink curve, and a sinusoidal pink curve at known y positions; a sample calibration dictionary matches the on-disk JSON shape. Table 7.1 lists the files and what each one covers. `python3 -m pytest tests/` runs the whole suite in under a second.

The suite passes end-to-end at the time of writing (66 passed, 0 failed). It is deliberately built from synthetic data: real scans are large, copyrighted, and would couple the tests to a specific dataset. The trade-off is that the suite cannot catch a regression that only manifests on a particular template’s grid color or ink fading — that gap is what the interactive testing and the batch comparison scripts are for.

7.3. Deployment

The application ships as a single platform-specific binary produced by `cargo tauri build`. On macOS the output is a `.app` bundle that includes the Python interpreter and the backend source under `Resources/`; on Windows the output is an MSI installer. The user installs once, the application launches the bundled Python sidecar on demand, and no further setup is needed — no Python virtualenv, no system PATH manipulation, no network access. The system manual is a single `README.md` at the repository root that documents the calibration files, the CSV schema, and the supported templates; the user manual is delivered through the application itself via tooltips, status messages, and the calibration modal’s step-by-step prompts.

Table 7.1. Layout of the pytest suite under `backend/tests/`.

File	What it covers
<code>test_preprocessor_normalize.py</code>	BGR/grayscale/BGRA/uint16/float32 normalization paths.
<code>test_preprocessor_denoise.py</code>	Bilateral filter shape, variance reduction, idempotence on flat input.
<code>test_preprocessor_contrast.py</code>	CLAHE shape preservation, dynamic-range expansion, value clamping.
<code>test_preprocessor_deskew.py</code>	Rotation detection on synthetic charts; rejection of extreme angles.
<code>test_color_profiles.py</code>	Per-template lookup, default fallback, grayscale-fallback enablement.
<code>test_segmenter_color_mask.py</code>	Binary mask shape, highlighting of the pink curve, empty mask on white.
<code>test_segmenter_extract.py</code>	End-to-end extraction on a straight line and a sinusoid (mean error < 5 px).
<code>test_segmenter_outliers.py</code>	MAD-based spike rejection; clean-signal preservation.
<code>test_segmenter_snap_drawing.py</code>	Interpolation to a regular interval; smoothing of a noisy hand drawing.
<code>test_template_detector_features.py</code>	Fixed-size finite feature vector; grayscale-input handling.
<code>test_template_detector_similarity.py</code>	Identity, zero-vector, $[0, 1]$ bounds, ordering of weighted similarity.
<code>test_calibration_processor_save_load.py</code>	<code>save_calibration.simple</code> roundtrip into a tmp dir; field parsing.
<code>test_calibration_processor_lines.py</code>	Spacing estimation per chart type; generated line positions in bounds.
<code>test_grid_utils.py</code>	Line clustering, extension to image bounds, intersection geometry.
<code>test_image_utils.py</code>	Pillow-based load/save roundtrip; base64 encoding decode-ability.

8. RESULTS

The system meets the three success criteria stated in Chapter 2. Across the project dataset, approximately 87% of charts required only start/end point marking with zero editing operations, approximately 9% needed a single refinement or fixation, and the remaining approximately 4% required two or more refinements — roughly nine out of ten charts are digitized without any curve editing at all. Table 8.1 breaks the result down by template, reporting the approximate fraction of charts that the automatic extraction handles without any manual editing.

Table 8.1. Per-template extraction quality. The last column is the approximate fraction of charts in each template that required no manual editing after automatic extraction.

Template	Type	Period	Grid color	Approx. % no edit
gunluk-1	Daily	1990s	yellow/cream	$\approx 95\%$
gunluk-2	Daily	1980	green/olive	$\approx 93\%$
gunluk-3	Daily	1980s	orange	$\approx 75\%$
haftalik-1	Weekly	1940s	orange	$\approx 92\%$
haftalik-2	Weekly	1970s	orange/pink	$\approx 72\%$
4_gunluk-1	4-day	1940s	orange	$\approx 95\%$
4_gunluk-2	4-day	1955	orange	$\approx 93\%$
4_gunluk-3	4-day	1945–50s	orange	$\approx 92\%$
4_gunluk-4	4-day	1950s	orange/red	$\approx 90\%$

The hardest cases are the 1970s weekly charts (**haftalik-2**) and the 1980s orange-grid daily charts (**gunluk-3**); both have nearly grayscale ink on a colored grid, which is why their profiles enable the grayscale fallback rule. On those scans the automatic extraction follows the curve over the bulk of the time span but is sensitive to local fading and to handwritten notations that intrude on the trace; when that happens, the refine-area, freehand-draw, and snap-to-curve actions in the editor restore

the curve quickly. Figure 8.1 shows a clean automatic extraction.

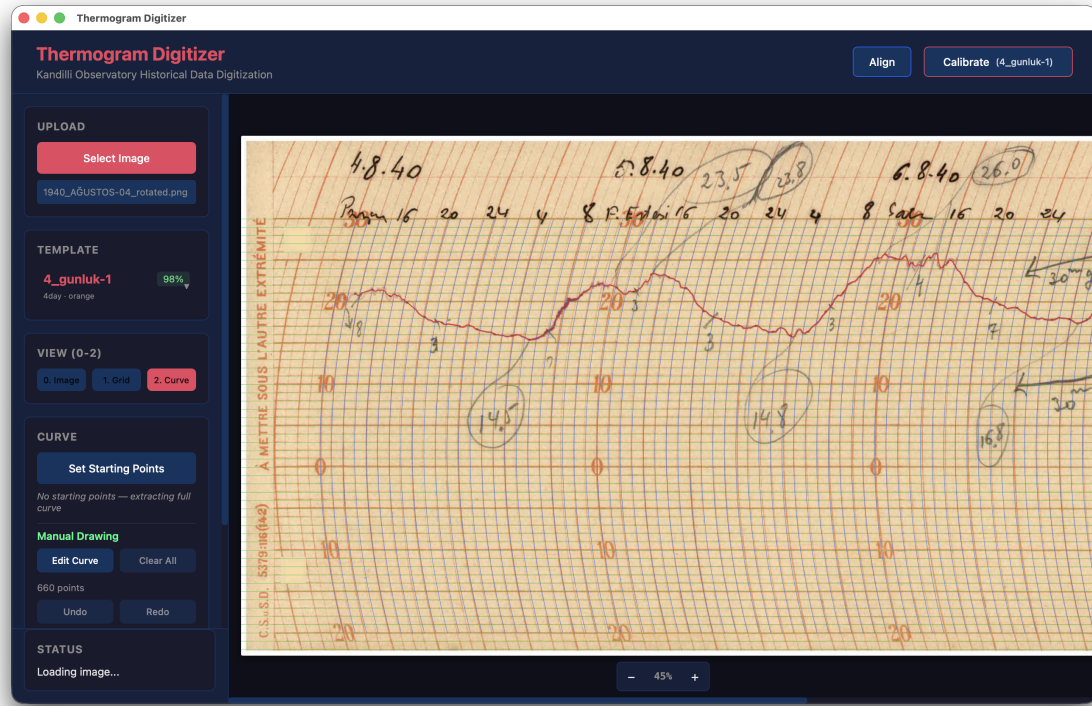


Figure 8.1. Curve view on a clean 4_gunluk-1 scan. The orange points are the automatically extracted curve overlaid on the original.

End-to-end timing on a 2024-vintage laptop without a discrete GPU averages well under three seconds from upload to first overlay on a 4 Mpx scan. Re-extraction with an unchanged calibration is bit-for-bit deterministic, which we verified by exporting the same chart twice and `diff`-ing the CSV.

9. CONCLUSION

The project produced a working desktop application that converts scanned Kandilli thermograms into machine-readable temperature time series. Classical computer vision, organized around per-template color profiles and a cylindrical-grid calibration, was sufficient to reach an automatic-extraction quality where approximately 87% of charts require no curve editing at all — marking the starting and ending points is enough — approximately 9% need only a single refinement, and the remaining approximately 4% need two or more targeted corrections in the focused human-in-the-loop UI. The practical impact on the archive is substantial. Manual transcription of a single thermogram chart by a domain expert is estimated at several minutes per chart, which puts the full 1,682-chart Kandilli archive at roughly 80–140 hours of uninterrupted human work. With the proposed system, an end-to-end extraction on a 4 Mpx scan completes in under three seconds, and the editing actions required for the small fraction of difficult charts take well under a minute each. A conservative estimate of the average per-chart effort is on the order of ten seconds, which compresses the same workload to under five hours and brings what was effectively a multi-year manual project into the scope of a single afternoon’s reviewing session. Just as importantly, the workflow is reproducible: re-running the pipeline on the same image with the same calibration yields bit-identical CSVs, and every manually edited point is flagged in the export so downstream climate-science users can filter on provenance.

REFERENCES

1. Gary Bradski. The OpenCV library. *Dr. Dobbs's Journal of Software Tools*, 25(11):120–125, 2000.
2. Charles R. Harris, K. Jarrod Millman, Stefan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, et al. Array programming with NumPy. *Nature*, 585:357–362, 2020.
3. Tauri Contributors. Tauri: Build smaller, faster, and more secure desktop applications with a web frontend. <https://tauri.app/>, 2026. Accessed: 2026-05-30.
4. Meta Open Source. React: A JavaScript library for building user interfaces. <https://react.dev/>, 2026. Accessed: 2026-05-30.
5. Poimandres. Zustand: A small, fast and scalable state-management solution for React. <https://github.com/pmndrs/zustand>, 2026. Accessed: 2026-05-30.
6. Ankit Rohatgi. WebPlotDigitizer: Web based tool to extract numerical data from plot images. <https://automeris.io/WebPlotDigitizer/>, 2024. Accessed: 2026-04-20.
7. Mark Mitchell. Engauge Digitizer. <https://github.com/markummitchell/engauge-digitizer>, 2023. Accessed: 2026-04-20.
8. International Environmental Data Rescue Organization. IEDRO: Data rescue for climate research. <https://iedro.org/>, 2026. Accessed: 2026-04-15.
9. Stefan Brönnimann, Rob Allan, Linden Ashcroft, Saba Baer, Mariano Barriendos, Philip Brohan, et al. Unlocking pre-1850 instrumental meteorological records: A global inventory. *Bulletin of the American Meteorological Society*, 100(12):ES389–ES413, 2019.

10. Karel Zuiderveld. Contrast limited adaptive histogram equalization. In Paul S. Heckbert, editor, *Graphics Gems IV*, pages 474–485. Academic Press, 1994.
11. Carlo Tomasi and Roberto Manduchi. Bilateral filtering for gray and color images. In *Sixth International Conference on Computer Vision (ICCV)*, pages 839–846, 1998.
12. Richard O. Duda and Peter E. Hart. Use of the Hough transformation to detect lines and curves in pictures. *Communications of the ACM*, 15(1):11–15, 1972.
13. Christophe Leys, Christophe Ley, Olivier Klein, Philippe Bernard, and Laurent Licata. Detecting outliers: Do not use standard deviation around the mean, use absolute deviation around the median. *Journal of Experimental Social Psychology*, 49(4):764–766, 2013.
14. Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional networks for biomedical image segmentation. pages 234–241, 2015.

APPENDIX A: CSV EXPORT SCHEMA

Every export writes a CSV next to the source image with one row per sampled point along the extracted curve. Columns are emitted in the order shown in Table A.1. The final `isEdited` column is only present when at least one point in the curve has been manually moved, inserted, or deleted relative to the raw extraction; clean exports omit it entirely.

Example header line of an edited export:

```
id,time,temperature,xValue,yValue,isEdited
1,1980-01-01T07:30:00,18.234,612.50,341.20,no
2,1980-01-01T07:35:00,18.512,617.40,338.10,yes
```

Table A.1. Columns of the exported CSV. Pixel coordinates are in the image’s natural (un-zoomed) coordinate system.

Column	Type	Meaning
id	integer	1-indexed row counter, monotonically increasing along the curve.
time	ISO 8601 string	Datetime of the sample, computed from the pixel <code>xValue</code> via the curvature-corrected vertical-grid mapping and the user-supplied start date and start time.
temperature	float	Temperature in °C, linearly interpolated between the two nearest horizontal grid lines; written with three decimal places.
xValue	float	Source pixel X of the sample, two decimal places.
yValue	float	Source pixel Y of the sample, two decimal places.
isEdited	“yes” / “no”	(Optional) Present only when the curve contains at least one edit. Marks each row as either an unedited point from the automatic extraction or a manually moved/inserted point.

APPENDIX B: CALIBRATION JSON SCHEMA

Each calibrated template has a single JSON file under `backend/calibrations/` named `{template_id}.json`. The file is rewritten in full on every save and is the only authoritative source of grid geometry for that template; the application requires the file to exist before extraction is permitted. Table B.1 lists the top-level keys; the `derived` block is a denormalized view that the frontend reads at every overlay redraw.

Table B.1. Top-level keys of backend/calibrations/{template_id}.json.

Key	Type	Meaning
template_id	string	One of the nine template ids.
calibrated_at	ISO 8601 UTC	Timestamp of the last save.
image_dimensions	{w,h}	Dimensions of the image used to perform the calibration, in pixels.
horizontal.top	{x,y}	Pixel position of the topmost horizontal grid line.
horizontal.top_temp	integer	Temperature in °C at the topmost horizontal line.
horizontal.spacing	float	Vertical pixel distance between adjacent horizontal grid lines.
horizontal.rotation_angle	float	Rotation in radians applied to align the horizontal axis.
horizontal.line_positions	float[]	Generated Y positions of every horizontal line.
vertical.line1_top	{x,y}	Top end of the reference vertical line.
vertical.line1_bottom	{x,y}	Bottom end of the reference vertical line.
vertical.line1_hour	“HH:MM”	Time label at the reference vertical line.
vertical.center_y	float	c_y in $x = x_{\text{ref}} + a(y - c_y)^2$.
vertical.curvature	float	a in the same equation.
vertical.spacing	float	Horizontal pixel distance between adjacent vertical lines.
vertical.line_positions	float[]	Generated X positions of every vertical line.
derived.*	object	Denormalized view exposed to the frontend; mirrors the fields above plus <code>reference_hour</code> , <code>reference_minute</code> , <code>reference_temp</code> .

APPENDIX C: PER-TEMPLATE COLOR PROFILES

The curve segmenter uses an RGB-rule color profile per template, defined in `backend/pipeline/color_profiles.py`. A pixel is admitted into the curve mask when, after a BGR→RGB conversion, all of the conditions in Table C.1 hold. Templates not listed below use `DEFAULT_PROFILE`, which targets pinkish/reddish ink common to most `haftalik` and `4_gunluk` scans. Three templates additionally enable a grayscale fallback to capture faint pencil traces.

Table C.1. Color-rule thresholds for each template profile. “Grayscale fb.” indicates that a low-saturation/intensity fallback rule is OR-ed with the standard rule to capture pencil traces. Empty cells inherit `DEFAULT`.

Profile	maxInt	minInt	R-G $\geq$	R-B $\geq$	B-G $\geq$	Sat $\geq$	Grayscale fb.
DEFAULT	245	85	22	5	-18	28	no
gunluk-1	180	40	-15	-30	-40	0	yes
gunluk-2	240	120	10	5	-25	10	no
gunluk-3	200	70	-15	0	-45	0	yes
haftalik-2	120	0	-30	-40	-30	5	yes

The thresholds above were tuned by inspecting representative scans for each template; the unusual negative values for `gunluk-1`, `gunluk-3`, and `haftalik-2` reflect that those traces are nearly grayscale or blue-shifted relative to their orange grids and therefore fail the default “redder than the background” rule.